

A^{-1}

El Papel

La Inversa de una Matriz

`inv(A)`

El Procesador

$$A^{-1} = \left(\frac{1}{|A|} \right) \times \text{Adj}(A)$$

```
numpy.linalg import inv  
x=inv(A)
```


La Base Algebraica

$$A \times A^{-1} = I$$

Matriz Original

El sistema de datos o transformaciones iniciales.

La **Matriz Inversa**

El objetivo algebraico. La transformación que deshace a la matriz original.

Matriz Identidad

El equivalente matricial al número 1. Una diagonal de unos rodeada de ceros.

El objetivo algebraico: Multiplicar una matriz por su inversa siempre resulta en la matriz Identidad. Es el paso fundamental para despejar incógnitas en el álgebra lineal pura.

Método algebraico tradicional

$$A^{-1} = \left(\frac{1}{|A|} \right) \times \text{Adj}(A)$$

1. Calcular la matriz de cofactores.

2. Transponer para obtener la matriz adjunta ($\text{Adj}(A)$).

3. Dividir cada elemento por el determinante ($|A|$).



**Elegante en el papel,
pero una trampa para
los procesadores.**

Este es el método que aprendemos en clase para matrices pequeñas, pero es la opción más ineficiente que una computadora podría utilizar para resolver sistemas reales.



Error de Redondeo

Las computadoras usan aritmética de punto flotante. Imprecisiones minúsculas se vuelven errores masivos.



Lentitud Algorítmica

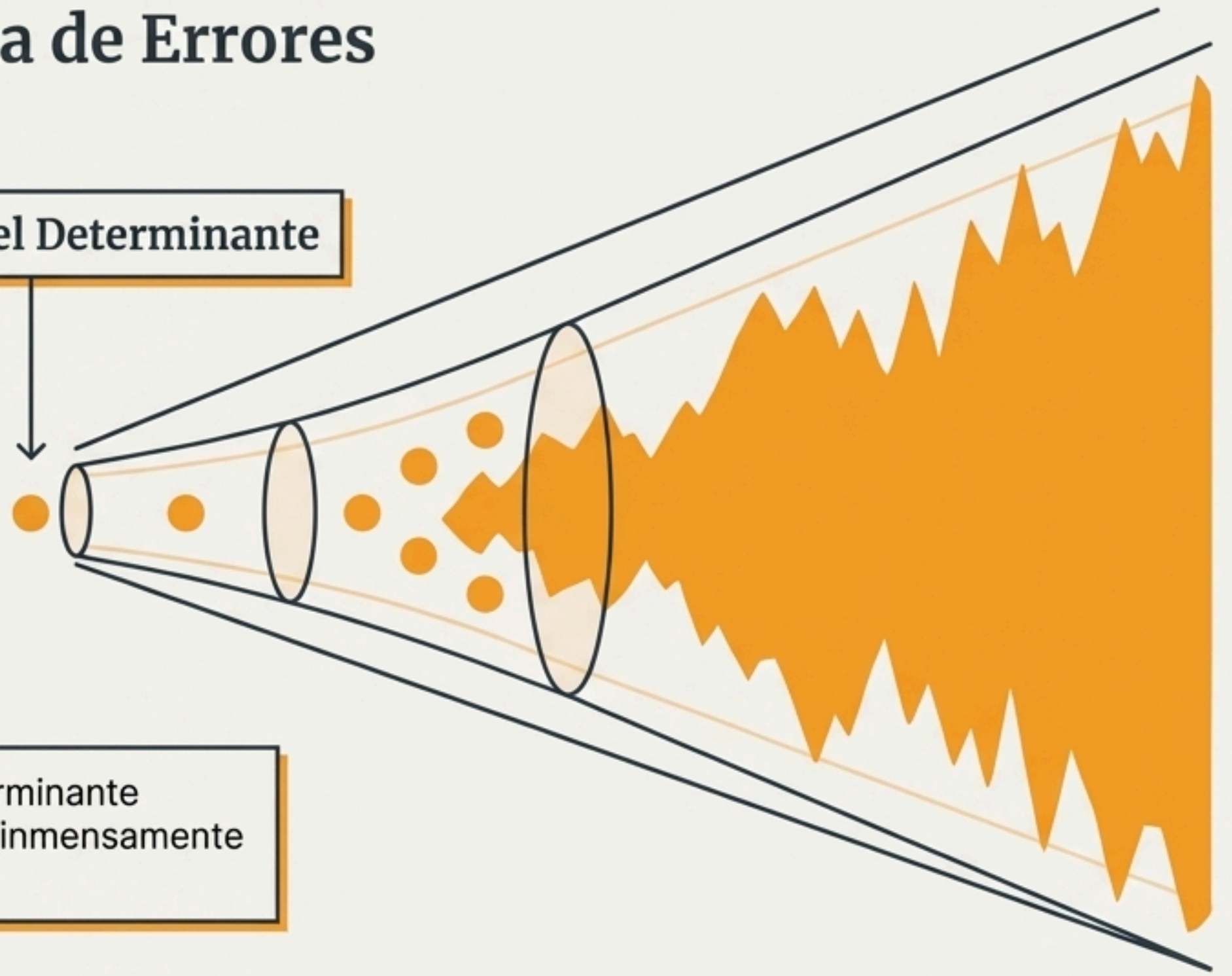
Calcular cofactores exige tiempo factorial ($O(n!)$). Para matrices grandes, el cálculo se vuelve físicamente inmanejable.

La Avalancha de Errores

El Trampa del Determinante

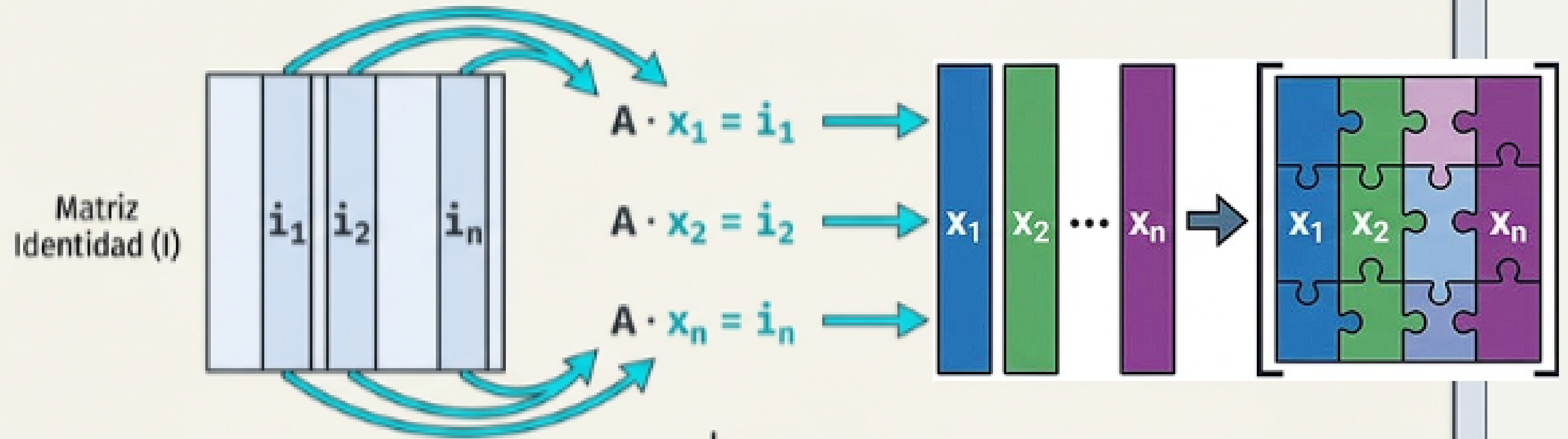
$\frac{1}{\text{casi cero}}$

Dividir por un determinante diminuto amplifica inmensamente las imprecisiones.



El Método Computacional: Resolviendo $AX = I$

El Solucionador por Columnas



La computadora asume que V es la Matriz Identidad (I). Resuelve el sistema iterativamente, columna por columna, para encontrar los vectores x . Al terminar y unirlos, la matriz resultante X es matemáticamente idéntica a A^{-1} .

Los Motores de Cálculo

`np.linalg.solve()`

Eliminación Gaussiana

Reducir la matriz original **A** a una forma escalonada.



Aplicar sustitución hacia atrás (Back-substitution).

Descomposición LU

Separar **A** en una matriz triangular Inferior (**L**) y Superior (**U**).



Resolver eficientemente múltiples columnas consecutivas.

¿Por qué usar esto? Ambos métodos reducen la matriz a una forma más simple, permitiendo resolver **X** de manera extraordinariamente rápida y con errores de redondeo rigurosamente controlados.

La Implementación en Código

```
from numpy.linalg import inv, solve

# El método directo (abstracto)
X = inv(A)
```



Abstracción de Alto Nivel:

Aunque numpy ofrece esta función de un solo paso, bajo la superficie no usa cofactores. Implementa silenciosamente las rutinas de Descomposición LU y resolución de $AX=I$ que hemos discutido.

La Paradoja de la Utilidad

~~A^{-1}~~

En problemas reales de física e ingeniería, casi nunca necesitamos calcular la matriz inversa explícitamente.

Si el objetivo final es resolver un sistema $Ax = v$ para encontrar el vector x , calcular primero A^{-1} es un desvío ineficiente. Es computacionalmente más rápido y preciso pasar $Ax = v$ directamente por la Eliminación Gaussiana o LU, sin llegar a construir la matriz inversa jamás.